

■ Aman

Hackathon · Fiche des Fonctionnalités

Chaque équipe choisit **autant de fonctionnalités qu'elle veut**. Les points s'accumulent.
Plusieurs équipes peuvent choisir la même fonctionnalité — c'est la **meilleure implémentation** qui gagne.
IA, librairies, tout est autorisé. Ce qui est jugé c'est le résultat et votre capacité à l'expliquer.

■ FACILE — 5 pts

#01 Mode Sombre / Clair

5

■ SITUATION

L'app est utilisée dehors, sous le soleil marocain, sur des téléphones avec des écrans pas toujours lisibles. Un utilisateur se plaint que l'interface trop sombre lui brûle les yeux la nuit, un autre ne voit rien en plein jour.

■ MISSION

Résolvez le problème de lisibilité de l'interface selon le contexte d'utilisation. L'utilisateur doit pouvoir choisir son mode et retrouver son choix à chaque ouverture de l'app.

■ CONTRAINTE

La solution doit s'appliquer à toute l'interface sans exception. Pas de zone qui reste dans l'ancien mode.

#02 Recherche dans la liste

5

■ SITUATION

Un technicien ONEE doit vérifier rapidement si une fuite dans un quartier précis a déjà été signalée. Il y a 40 signalements dans la liste. Il n'a pas le temps de tout lire.

■ MISSION

Permettez-lui de trouver ce qu'il cherche en tapant quelques lettres. La recherche doit fonctionner sur la ville, le quartier et la description.

■ CONTRAINTE

Le résultat doit apparaître pendant que l'utilisateur tape. Pas de bouton 'Rechercher'. Gérez le cas où aucun résultat ne correspond.

#03 Bouton Confirmer la panne

5

■ SITUATION

Un habitant voit un signallement dans sa rue. Il ne veut pas créer un doublon — il veut juste dire 'moi aussi je confirme'. Actuellement, soit il crée un doublon inutile, soit il ne fait rien.

■ MISSION

Donnez-lui un moyen de confirmer une panne existante. Plus un signallement a de confirmations, plus il devient crédible et visible.

■ CONTRAINTE

Quelqu'un ne doit pas pouvoir confirmer le même signallement deux fois. Définissez à partir de combien de confirmations le signallement change d'apparence.

#04 Traduction Darija

5

■ SITUATION

Une grande partie des habitants des quartiers populaires marocains s'exprime en darija, pas en français formel. Les descriptions de pannes dans l'app sonnent faux — personne ne parle comme ça dans la rue.

■ MISSION

Rendez l'app plus marocaine. Ajoutez la possibilité de lire les descriptions en darija.

■ CONTRAINTE

Pas d'API externe. Construisez une table de traduction de 10 phrases minimum. Si une description n'a pas de traduction, l'app ne doit pas planter.

■■ MOYEN — 10 pts

#05 Tri des signalements

10

■ SITUATION

La liste affiche les signalements dans l'ordre d'arrivée. Mais une mère de famille veut voir les pannes les plus graves en premier. Un journaliste veut les plus récentes. Un technicien veut celles qui durent depuis le plus longtemps.

■ MISSION

Donnez aux utilisateurs le contrôle sur l'ordre d'affichage. Proposez au moins 3 critères de tri pertinents pour ce contexte.

■ CONTRAINTE

Le tri ne doit pas modifier les données originales. Il doit persister quand l'utilisateur revient à la liste après avoir changé de vue.

#06 Sauvegarde des filtres dans l'URL

10

■ SITUATION

Un militant associatif filtre l'app sur Casablanca + pannes actives et trouve 4 cas alarmants. Il veut envoyer ce qu'il voit à un journaliste par WhatsApp. Mais quand le journaliste ouvre le lien, il voit la vue par défaut.

■ MISSION

Le lien partagé doit reconstituer exactement la même vue avec les mêmes filtres actifs.

■ CONTRAINTE

Sans backend, sans compte, sans base de données. Tout doit fonctionner uniquement via l'URL.

#07 Bannière d'alerte critique

10

■ SITUATION

Trois quartiers sont sans eau depuis 48h. C'est une urgence. Mais dans l'app, ces signalements se noient dans la liste comme n'importe quel autre. Un signalement banal résolu hier prend autant de place.

■ MISSION

Les situations critiques doivent forcer l'attention. Concevez un système d'alerte visible qui s'active automatiquement quand la situation est grave.

■ CONTRAINTE

Définissez précisément les conditions qui déclenchent l'alerte. Trop sensible = tout devient urgent. La bannière doit pouvoir être fermée. Elle doit se réactiver si une nouvelle panne critique apparaît.

#08 Suivi de durée des pannes

10

■ SITUATION

Un habitant voit un signalement actif. Il veut savoir depuis combien de temps exactement les gens sont sans eau. 'Il y a 3 heures' ne lui dit pas grand chose. 48h sans eau c'est une urgence humanitaire. 2h c'est une gêne.

■ MISSION

Affichez la durée réelle de chaque panne active. Cette durée doit changer d'apparence pour signaler quand la situation devient critique.

■ CONTRAINTE

La durée doit se mettre à jour en temps réel, pas seulement au chargement de la page. Définissez vos propres seuils (longue / critique) et justifiez-les en démo.

#09 Signalement de faux rapports

10

■ SITUATION

N'importe qui peut soumettre n'importe quoi dans l'app. Un concurrent malveillant pourrait créer de faux signalements pour paniquer les habitants d'un quartier ou nuire à une commune.

■ MISSION

La communauté doit pouvoir contester un signalement qu'elle juge faux. Le système doit réagir de façon proportionnée et visible.

■ CONTRAINTE

Un seul flag ne doit pas suffire à faire disparaître un signalement — trop facile à abuser. Mais un signalement très contesté ne doit pas rester affiché normalement. Trouvez l'équilibre.

#10 Abonnement aux alertes

10

■ SITUATION

Un utilisateur ne veut pas ouvrir l'app toutes les heures. Il veut être alerté quand ça le concerne. Une panne à Tanger ne l'intéresse pas s'il habite à Agadir.

■ MISSION

Concevez un système d'abonnement aux alertes locales. L'utilisateur choisit sa ville et laisse son contact. L'interface doit le convaincre que le système est réel et fiable.

■ CONTRAINTE

Pas de vraies notifications push. Mais l'expérience doit créer la confiance. Les abonnements disparaissent-ils au refresh ? Justifiez votre choix de persistance.

■■■ AVANCÉ — 20–25 pts

#11 Recherche de quartier sur la carte

20

■ SITUATION

Un utilisateur ouvre la carte et voit des dizaines de marqueurs. Il veut trouver les pannes près de chez lui mais il ne connaît pas les coordonnées GPS de son quartier. Il ne sait pas zoomer sur la bonne zone.

■ MISSION

Connectez la recherche textuelle à la carte. Quand l'utilisateur tape un quartier ou une ville, la carte doit se déplacer automatiquement vers cette zone et mettre en évidence les signalements correspondants.

■ CONTRAINTE

La solution doit rester dans la vue carte. Gérez le cas où le quartier tapé n'existe pas dans les données.

#12 Rayon de zone affectée

20

■ SITUATION

L'app montre un point sur la carte pour chaque panne. Mais une panne d'eau ne touche pas un point — elle touche une zone. Le voisin à 300 mètres ne sait pas s'il est concerné.

■ MISSION

Visualisez l'étendue probable d'une panne autour de chaque marqueur. La zone doit être proportionnelle à la gravité du signalement.

■ CONTRAINTE

L'utilisateur ne peut pas dessiner la zone manuellement — tout doit être automatique. Le cercle doit se mettre à jour si la sévérité change.

#13 Upload de photos

20

■ SITUATION

Un signalement texte peut être inventé. Une photo prouve que la panne est réelle — eau trouble, rue inondée, robinet sec filmé. C'est la preuve la plus simple et la plus convaincante.

■ MISSION

Permettez d'ajouter une photo en option lors d'un signalement. La photo doit être visible dans la fiche du signalement.

■ CONTRAINTE

Gérez les fichiers trop volumineux. L'app ne doit pas planter avec une image de 10MB. Définissez une limite et expliquez-la.

#14 Ticker temps réel

20

■ SITUATION

L'app est statique — rien ne bouge. Un utilisateur qui la laisse ouverte ne sait pas si quelque chose s'est passé. L'urgence ne transparaît pas.

■ MISSION

Créez un sentiment d'activité en temps réel. Les nouveaux signalements et les résolutions récentes doivent apparaître de façon visible sans que l'utilisateur ait à rafraîchir.

■ CONTRAINTE

La solution ne doit pas prendre trop de place dans l'interface. Elle doit apparaître naturellement sans gêner la navigation.

#15 Classement des villes

20

■ SITUATION

Un responsable régional veut savoir en un coup d'oeil quelle ville souffre le plus en ce moment. Il ne veut pas lire 40 signalements — il veut une réponse immédiate.

■ MISSION

Créez une vue synthétique qui classe les villes par gravité de la situation. La réponse doit être visible en moins de 3 secondes.

■ CONTRAINTE

Le classement doit se mettre à jour automatiquement quand un nouveau signalement est ajouté. Réfléchissez à ce que 'gravité' veut dire — est-ce juste le nombre, ou la sévérité compte aussi ?

#16 Mini graphiques SVG

20

■ SITUATION

Les chiffres dans la barre de stats sont froids et statiques. Un utilisateur ne sait pas si 5 pannes actives c'est beaucoup ou peu. Est-ce qu'on est dans une mauvaise semaine ou une bonne ?

■ MISSION

Remplacez les chiffres bruts par une visualisation qui donne du contexte. L'utilisateur doit comprendre la tendance d'un coup d'oeil.

■ CONTRAINTE

Vous n'avez pas de vraies données historiques — vous devez les générer de façon crédible. Expliquez en démo pourquoi votre représentation est honnête malgré ça.

#17 Traduction complète FR / AR

25

■ SITUATION

L'app est entièrement en français. Mais une grande partie des Marocains les plus touchés par les pannes d'eau — quartiers populaires, zones rurales — lisent et écrivent en arabe.

■ MISSION

Rendez l'app entièrement utilisable en arabe. Chaque texte visible dans l'interface doit avoir sa traduction. Le sens de lecture doit s'inverser.

■ CONTRAINTE

Vous devez gérer le RTL (droite à gauche) sans casser la mise en page existante. Testez sur un vrai contenu arabe, pas juste quelques mots.

#18 Flux de résolution des pannes

25

■ SITUATION

Une panne est résolue. L'eau revient. Mais le signalement reste 'actif' sur l'app pendant des jours. Les habitants font confiance à l'app et s'inquiètent inutilement. La désinformation par inaction.

■ MISSION

Concevez un flux clair pour marquer une panne comme résolue. L'interface doit se mettre à jour immédiatement et le changement doit être visible partout — liste et carte.

■ CONTRAINTE

N'importe qui peut marquer une panne comme résolue, pas seulement celui qui l'a signalée. Comment gérez-vous le risque que quelqu'un ferme une panne encore active par erreur ou malveillance ?

■■■■■ DIFFICILE — 35 pts

#19 Clustering des marqueurs

35

■ SITUATION

Quand on dézoomé pour voir tout le Maroc, les marqueurs se superposent et la carte devient illisible. Un journaliste qui veut faire un screenshot pour son article ne voit qu'un tas de points colorés.

■ MISSION

Regroupez les marqueurs proches en clusters lisibles quand la carte est dézoomée. En zoomant, les clusters s'ouvrent pour révéler les signalements individuels.

■ CONTRAINTE

La couleur d'un cluster doit refléter la panne la plus grave qu'il contient. Les clusters doivent se mettre à jour quand un nouveau signalement est ajouté.

#20 Panel Admin sécurisé

35

■ SITUATION

L'app grandit. Il y a des faux signalements, des doublons, des pannes marquées actives depuis 2 semaines. Quelqu'un doit pouvoir nettoyer tout ça. Mais cette personne ne doit pas être n'importe qui.

■ MISSION

Construisez un outil de gestion pour les administrateurs. Pas juste un panneau avec des boutons — un vrai outil pensé pour quelqu'un qui l'utilise tous les jours.

■ CONTRAINTE

L'accès doit être protégé. Dans un contexte sans backend, définissez ce que 'protégé' veut dire et assumez les limites de votre approche en démo.

■■■■■■ EXPERT — 50-60 pts

#21 Heatmap des pannes

50

■ SITUATION

La carte montre des points individuels. Mais un décideur politique veut voir les zones de concentration — les 'points chauds' où les problèmes d'eau se répètent. Il ne veut pas compter les marqueurs.

■ MISSION

Superposez une heatmap sur la carte Leaflet existante. Les zones à forte densité de pannes doivent apparaître clairement en rouge/orange, les zones calmes en vert/transparent.

■ CONTRAINTE

L'intensité doit refléter la sévérité, pas juste le nombre. La heatmap doit se mettre à jour quand un nouveau signalement est ajouté. Elle doit pouvoir être activée/désactivée sans recharger la page.

#22 Simulation temps réel

50

■ SITUATION

L'app est présentée avec 10 signalements statiques. Pour montrer sa valeur lors d'une démo à un partenaire institutionnel, il faudrait voir l'app 'vivre' — de nouvelles pannes qui apparaissent, des résolutions qui arrivent.

■ MISSION

Créez un mode simulation qui génère automatiquement de l'activité réaliste. Les pannes simulées doivent apparaître dans les bonnes villes marocaines, avec des données crédibles.

■ CONTRAINTE

La simulation doit pouvoir être démarrée et arrêtée. Les pannes générées ne doivent pas avoir des coordonnées hors du Maroc. La fréquence doit être contrôlable.

#23 Prédiction des pannes

60

■ SITUATION

Les pannes d'eau ne sont pas aléatoires — certaines zones sont plus vulnérables, certaines périodes plus risquées. Un responsable municipal voudrait anticiper plutôt que réagir.

■ MISSION

Créez un système qui attribue un score de risque à chaque zone et estime un temps de rétablissement probable. Ce système doit être basé sur une logique explicable.

■ CONTRAINTE

Vous n'avez pas de vraies données historiques. Votre algorithme doit quand même être défendable. Vous devez être capables d'expliquer chaque facteur pris en compte. Présentez-le comme une estimation, pas une certitude.

#24 Replay chronologique

60

■ SITUATION

L'app montre l'état actuel. Mais comment la situation a-t-elle évolué depuis ce matin ? Depuis une semaine ? Un chercheur ou un journaliste voudrait revoir l'évolution des pannes dans le temps.

■ MISSION

Créez une timeline interactive qui permet de 'rejouer' l'évolution des pannes. L'utilisateur glisse un curseur dans le temps et la carte se met à jour pour montrer l'état à ce moment-là.

■ CONTRAINTE

La timeline doit pouvoir être mise en pause. La carte doit correspondre exactement à l'état au moment sélectionné. Proposez au moins deux vitesses de lecture.

#25 Support Hors-ligne / PWA

60

■ SITUATION

Dans les zones rurales marocaines et les quartiers périphériques — là où les pannes d'eau sont les plus fréquentes — la connexion internet est souvent instable. L'app cesse de fonctionner exactement là où elle est le plus utile.

■ MISSION

Transformez l'app en Progressive Web App fonctionnelle hors-ligne. Elle doit pouvoir être installée sur un téléphone et afficher les dernières données connues sans connexion.

■ CONTRAINTE

Montrez que l'app est installable depuis le navigateur. En coupant le wifi, l'app doit afficher les dernières données connues avec un indicateur clair du mode hors-ligne.

■ Points Bonus

UI / UX exceptionnelle

Interface soignée, cohérente, agréable.

+10 pts

Animations fluides

Transitions qui apportent de la valeur.

+5 pts

Responsive mobile

Fonctionne sur un écran de téléphone.

+10 pts

Accessibilité

Utilisable au clavier, contrastes suffisants.

+5 pts

Fonctionnalité créative supplémentaire

Feature originale non listée, bien justifiée.

+15 pts

Meilleure présentation

Démo claire, convaincante, bien préparée.

+10 pts

■ Règles

Pas de limite de fonctionnalités

Prenez autant que vous voulez. Les points s'accumulent.

Doublons autorisés

Plusieurs équipes peuvent faire la même feature. La meilleure version gagne.

Codebase existant

Vous travaillez sur Aman. Votre code doit s'intégrer proprement.

Tout outil autorisé

IA, librairies npm, tout est permis.

Démo : 4 minutes max

Montrez chaque feature. Expliquez vos choix. Répondez aux questions.

■ Critères

Utilité

Ça résout vraiment le problème posé ?

Intégration

Ça s'intègre naturellement dans Aman ?

Décisions

L'équipe justifie ses choix techniques ?

Qualité

Le code fonctionne, l'UI est propre.